Playwright – Workers, Parallel Execution, Serial Mode and `forbidOnly`

1. Workers in Playwright

Workers decide **how many tests Playwright can execute at the same time**.

Example:

```
export default defineConfig({
  workers: 4
});
```

Here:

```
workers = 4
```

means Playwright can use **maximum 4 worker processes** for test execution.

Think like this:

```
Worker 1 → Test/File
Worker 2 → Test/File
Worker 3 → Test/File
Worker 4 → Test/File
```

More workers can make execution faster.

---

2. `fullyParallel`

We can configure:

```
fullyParallel: true
```

or

New Concept: How Workers Receive Work

The key idea

workers control how many tasks may execute at the same time. **fullyParallel** controls the size of the task Playwright gives to a worker.

1. What is a worker?

A worker is a separate operating-system process started by Playwright. Each worker has its own browser instance and runs one assigned unit of work at a time. When that unit finishes, the worker becomes idle and asks the scheduler for the next available unit.

Setting	Unit placed in the queue	What an idle worker can take
fullyParallel: false	A complete spec file	Another spec file
fullyParallel: true	An individual test	The next available test, even from the same spec file

Important: **workers are not permanently attached to a particular file or test.** They take available work, finish it, and then take more work.

Example configuration

```
export default defineConfig({
  workers: 3,
  fullyParallel: false
});
```

Here, at most three worker processes can run at once. The next page shows exactly what they receive.

Step-by-Step: fullyParallel = false

The scheduler distributes work **by spec file**. One worker receives one complete spec file, and the tests inside that file run one after another by default.

Example suite

login.spec.ts: Login A, Login B, Login C
checkout.spec.ts: Checkout A, Checkout B
payment.spec.ts: Payment A, Payment B
invoice.spec.ts: Invoice A

Time	Worker 1	Worker 2	Worker 3	Queue
Step 1	login.spec.ts Login A	checkout.spec.ts Checkout A	payment.spec.ts Payment A	invoice.spec.ts
Step 2	same file Login B	same file Checkout B	same file Payment B	invoice.spec.ts
Step 3	same file Login C	file finished **takes invoice.spec.ts**	file finished idle	empty

What should the learner notice?

1. Worker 1 cannot hand **Login B** to Worker 3 while login.spec.ts is still running.
2. The file is the scheduling unit, so tests in that file remain together.
3. When Worker 2 finishes checkout.spec.ts, it takes the next **file**: invoice.spec.ts.
4. A free worker may stay idle when no unassigned spec file remains.

Action: run and observe

```
npx playwright test --workers=3
```

Add **console.log(test.info().workerIndex)** inside each test to see which worker ran it.

Step-by-Step: fullyParallel = true

The scheduler can distribute work **by individual test**. Tests from inside the same spec file may run on different workers, provided they are not inside a serial group.

Example suite

shop.spec.ts: Login, Search, Add to cart, Checkout, Payment, Invoice
Configuration: **workers: 3**, **fullyParallel: true**

Time	Worker 1	Worker 2	Worker 3	Waiting tests
Step 1	Login	Search	Add to cart	Checkout, Payment, Invoice
Step 2	Login still running	Search **finished takes Checkout**	Add to cart still running	Payment, Invoice
Step 3	Login **finished takes Payment**	Checkout running	Add to cart **finished takes Invoice**	empty

The idle-worker rule

As soon as any worker finishes its current test, it can take the next available test from the queue. It does **not** need to wait for the other tests from that spec file to finish.

Try it yourself

1. Put six independent tests in one spec file.
2. Add a different short delay to each test so they finish at different times.
3. Run with **--workers=3** and **fullyParallel: true**.
4. Print the worker index and test title.
5. Observe that the first worker to finish takes the next waiting test.

Safe usage rule

Use fully parallel execution only when tests are independent. Do not share mutable accounts, page objects, files, or database state. If order matters, redesign the scenario or use serial mode intentionally.

Remember: fullyParallel: false gives workers whole spec files; fullyParallel: true lets idle workers take individual tests.

```
fullyParallel: false
```

`fullyParallel: false`

This is the normal/default behaviour.

```
fullyParallel: false
```

Playwright can run **different spec files in parallel**, but tests inside the same spec file normally run one after another.

Example:

```
login.spec.js

test1
test2
test3
```

Execution inside this file:

```
test1
  ↓
test2
  ↓
test3
```

But if we have multiple files:

```
login.spec.js
checkout.spec.js
payment.spec.js
invoice.spec.js
```

different workers can execute these files at the same time.

Example:

```
Worker 1 → login.spec.js
Worker 2 → checkout.spec.js
```

```
Worker 3 → payment.spec.js
Worker 4 → invoice.spec.js
```

So remember:

```
fullyParallel = false

Files → can run parallel
Tests inside same file → normally sequential
```

---

3. `fullyParallel: true`

Configuration:

```
export default defineConfig({
  workers: 4,
  fullyParallel: true
});
```

Now Playwright allows tests inside spec files also to run in parallel.

Example:

```
test('Login', async ({ page }) => {

});

test('Checkout', async ({ page }) => {

});

test('Payment', async ({ page }) => {

});

test('Invoice', async ({ page }) => {

});
```

With:

```
workers: 4,
fullyParallel: true
```

execution can look like:

```
Worker 1 → Login
Worker 2 → Checkout
Worker 3 → Payment
Worker 4 → Invoice
```

So remember:

```
fullyParallel = true

Files → parallel
Tests inside files → can also run parallel
```

---

4. Should We Always Use `fullyParallel: true`?

No.

It depends on the use case.

Suppose we have:

```
spec.js

test1 → Login
test2 → Checkout
test3 → Payment
test4 → Check Invoice
```

If every test is completely independent, parallel execution is fine.

But if Test 2 depends on Test 1, Test 3 depends on Test 2, etc., running them in parallel can create problems.

Ideally, automation tests should be **independent**.

A better design can be:

```
spec2.js

test1:
Login
↓
Checkout
↓
Payment
↓
Invoice
```

Another test:

```
test2:
Login
↓
Checkout
↓
Wrong Payment
↓
Verify Error
```

Now each test represents one complete business scenario.

Example:

```
test('Successful order flow', async ({ page }) => {

  // login

  // checkout

  // payment

  // verify invoice
});


test('Payment failure flow', async ({ page }) => {

  // login

  // checkout
```

```
  // enter wrong payment details

  // verify error
});
```

These two tests can safely run independently.

---

5. Changing Parallel Behaviour for One Spec File

Sometimes global configuration is parallel, but for one particular group/file we want different behaviour.

We can use:

```
test.describe.configure({
  mode: 'serial'
});
```

Example:

```
import { test, expect } from '@playwright/test';

test.describe.configure({
  mode: 'serial'
});

test('Login', async ({ page }) => {

});

test('Checkout', async ({ page }) => {

});

test('Payment', async ({ page }) => {

});

test('Invoice', async ({ page }) => {

});
```

Now tests in this group execute in order.

```
Login
 ↓
Checkout
 ↓
Payment
 ↓
Invoice
```

---

6. What Is `serial` Mode?

Serial means:

> Tests run one after another.

Example:

```
Test 1 → PASS
Test 2 → FAIL
Test 3 → SKIPPED
Test 4 → SKIPPED
```

Suppose:

```
test.describe.configure({
  mode: 'serial'
});
```

and we have:

```
Test 1 → Login
Test 2 → Checkout
Test 3 → Payment
Test 4 → Invoice
```

If Checkout fails:

```
Login     → PASS
Checkout  → FAIL
```

```
Payment   → SKIPPED
Invoice   → SKIPPED
```

Why?

Because Playwright assumes tests in a serial group may depend on the previous tests.

---

7. What Happens During Retry in Serial Mode?

Suppose:

```
retries: 1
```

and Test 2 fails.

First execution:

```
Test 1 → PASS
Test 2 → FAIL
Test 3 → SKIPPED
Test 4 → SKIPPED
```

During retry, Playwright retries the **whole serial group together**.

```
Test 1 → RUN AGAIN
Test 2 → RUN AGAIN
Test 3 → RUN
Test 4 → RUN
```

So an important interview/class point is:

> In serial mode, if one test fails, later tests are skipped, and during retry the serial group is retried together.

---

8. Parallel vs Serial

Normal Execution

```
fullyParallel: false
```

```
Spec files → parallel
Tests inside same file → sequential
```

Fully Parallel

```
fullyParallel: true
```

```
Spec files → parallel
Tests → parallel
```

Serial

```
test.describe.configure({
  mode: 'serial'
});
```

```
Test 1
 ↓
Test 2
 ↓
Test 3
 ↓
Test 4
```

One fails → remaining tests in that serial group are skipped.

---

9. Important Best Practice

Avoid creating dependent tests like:

```
test1 → Login

test2 → Checkout

test3 → Payment

test4 → Invoice
```

because:

```
test2 depends on test1
test3 depends on test2
test4 depends on test3
```

Instead create complete independent scenarios.

Example:

```
Test 1

Login
↓
Checkout
↓
Payment
↓
Invoice
```

```
Test 2

Login
↓
Checkout
↓
Invalid Payment
↓
Verify Error
```

Now Test 1 and Test 2 are independent and can run in parallel.

---

10. `forbidOnly`

Sometimes while debugging we write:

```
test.only('Login Test', async ({ page }) => {

});
```

`test.only()` means:

> Run only this selected test/test group and ignore the others.

Example:

```
test('Test 1', async () => {

});

test.only('Test 2', async () => {

});

test('Test 3', async () => {

});
```

Execution:

```
Test 1 → Not Run
Test 2 → RUN
Test 3 → Not Run
```

---

11. `forbidOnly: false`

Configuration:

```
forbidOnly: false
```

This means Playwright allows `test.only()`.

So if we have:

```
test.only('Login Test', async ({ page }) => {

});
```

Playwright will run the selected `only` test.

If multiple tests have `.only`:

```
test.only('Login', async () => {

});

test.only('Checkout', async () => {

});
```

both selected tests will run.

---

12. `forbidOnly: true`

Configuration:

```
forbidOnly: true
```

This is normally useful in CI/CD.

If somebody accidentally commits:

```
test.only(...)
```

Playwright will fail the run instead of silently running only that test.

This prevents a situation where we have:

```
100 tests
```

but because somebody accidentally wrote:

```
test.only(...)
```

CI executes only:

```
1 test
```

---

13. Recommended Configuration

Normally we use:

```
forbidOnly: !!process.env.CI
```

Meaning:

```
Local Machine
process.env.CI → normally false
forbidOnly → false
```

So developers/testers can use:

```
test.only()
```

for debugging.

But in CI:

```
process.env.CI → true
forbidOnly → true
```

So `.only` is not allowed.

This helps prevent accidental partial test execution.

---

14. Simple Example Configuration

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  workers: 4,

  fullyParallel: true,

  forbidOnly: !!process.env.CI,

  retries: process.env.CI ? 2 : 0

});
```

Meaning:

```
workers: 4
→ Maximum 4 workers can execute tests.

fullyParallel: true
→ Tests can execute in parallel.

forbidOnly: !!process.env.CI
→ test.only() allowed locally but blocked in CI.

retries
→ Retry failed tests in CI.
```

Quick Revision

```
workers
        ↓
How many worker processes can run tests simultaneously.


fullyParallel: false
        ↓
Different spec files can run parallel.
Tests in the same file normally run sequentially.
```

```
fullyParallel: true
        ↓
Tests can also run in parallel.


serial
        ↓
Tests run one after another.
One fails → following tests are skipped.
On retry → serial group runs again.


test.only()
        ↓
Run only selected test(s).


forbidOnly: false
        ↓
test.only() is allowed.


forbidOnly: true
        ↓
test.only() is not allowed.
Test run fails if `.only` is found.


forbidOnly: !!process.env.CI
        ↓
Local → .only allowed
CI → .only blocked
```

One Line to Remember

Workers decide HOW MANY tests can run simultaneously, `fullyParallel` decides WHERE parallelism is allowed, `serial` forces ordered execution, and `forbidOnly` protects CI from accidental `test.only()`.